Principles of Programming Languages Lecture 11: Paradigms: Functional programming.

Andrei Arusoaie[1]

[1]Department of Computer Science

December 20, 2016

Outline

Outline

Outline

Paradigms

- Imperative Programming: ✓

Paradigms

- Imperative Programming: ✓
- Object Oriented Programming: ✓

Paradigms

- Imperative Programming: ✓
- Object Oriented Programming: ✓
- Functional Programming

Paradigms

- Imperative Programming: ✓
- Object Oriented Programming: ✓
- Functional Programming
- Logic Programming

General facts about PLs

General facts about PLs

- Computational model based on: *state* + *modifiable variable* + *assignment*

General facts about PLs

- Computational model based on: *state* + *modifiable variable* + *assignment*
- *Computation proceeds by modifying values stored in locations*

General facts about PLs

- Computational model based on: *state* + *modifiable variable* + *assignment*
- *Computation proceeds by modifying values stored in locations*
- The *von Neumann Machine*

General facts about PLs

- Computational model based on: *state* + *modifiable variable* + *assignment*
- *Computation proceeds by modifying values stored in locations*
- The *von Neumann Machine*
- This is not the only possible model upon which to base a PL

General facts about PLs

- Computational model based on: *state* + *modifiable variable* + *assignment*
- *Computation proceeds by modifying values stored in locations*
- The *von Neumann Machine*
- This is not the only possible model upon which to base a PL:
  - It is possible to compute without referring to a state

General facts about PLs

- Computational model based on: *state* + *modifiable variable* + *assignment*
- *Computation proceeds by modifying values stored in locations*
- The *von Neumann Machine*
- This is not the only possible model upon which to base a PL:
  - It is possible to compute without referring to a state. How?

General facts about PLs

- Computational model based on: *state* + *modifiable variable* + *assignment*
- *Computation proceeds by modifying values stored in locations*
- The *von Neumann Machine*
- This is not the only possible model upon which to base a PL:
  - It is possible to compute without referring to a state. How?
  - ... by *rewriting* expressions

General facts about PLs

- Computational model based on: *state* + *modifiable variable* + *assignment*
- *Computation proceeds by modifying values stored in locations*
- The *von Neumann Machine*
- This is not the only possible model upon which to base a PL:
  - It is possible to compute without referring to a state. How?
  - ... by *rewriting* expressions
  - The computation is expressed in terms of modifications of the environment

Functional programming: history + characteristics

- This paradigm is as old as the imperative one

Functional programming: history + characteristics

- This paradigm is as old as the imperative one
- Turing machine...

Functional programming: history + characteristics

- This paradigm is as old as the imperative one
- Turing machine...
- ..... and λ-calculus

Functional programming: history + characteristics

- This paradigm is as old as the imperative one
- Turing machine...
- ..... and λ-calculus
- Ingredients: higher-order functions + recursion

Functional programming: history + characteristics

- This paradigm is as old as the imperative one
- Turing machine...
- ..... and λ-calculus
- Ingredients: higher-order functions + recursion
- PLs: Haskell, Miranda (pure) + Lisp, ML, Scheme, ...

Some mathematical background

- Math function: $f(x) = x^2$

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4
- But mathematicians say that they *define* $f(x)$

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4
- But mathematicians say that they *define* $f(x)$
- Indeed the *name* of the function is f

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4
- But mathematicians say that they *define* $f(x)$
- Indeed the *name* of the function is f
- . . . but we distinguish the *definition* from the *application*

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4
- But mathematicians say that they *define* $f(x)$
- Indeed the *name* of the function is f
- . . . but we distinguish the *definition* from the *application*, respectively:
  - f has a *formal* parameter x which indicates the *transformation* that f applies to arguments x

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4
- But mathematicians say that they *define* $f(x)$
- Indeed the *name* of the function is f
- . . . but we distinguish the *definition* from the *application*, respectively:
  - f has a *formal* parameter x which indicates the *transformation* that f applies to arguments x
  - $f(x)$ is the expression that results after the transformation happens

Some mathematical background

- Math function: $f(x) = x^2$, where $f : \mathbb{R} \to \mathbb{R}$
- Math function application: $f(2)$, with result 4
- But mathematicians say that they *define* $f(x)$
- Indeed the *name* of the function is f
- . . . but we distinguish the *definition* from the *application*, respectively:
  - f has a *formal* parameter x which indicates the *transformation* that f applies to arguments x
  - $f(x)$ is the expression that results after the transformation happens
- Functions = *expressible* values

More about writing function application

- Classical: $f(2)$

More about writing function application

- Classical: $f(2)$
- Available: $(f\ 2)$ or simply $f\ 2$

More about writing function application

- Classical: $f(2)$
- Available: $(f\ 2)$ or simply $f\ 2$
- The latter notation has an advantage:

More about writing function application

- Classical: $f(2)$
- Available: $(f\ 2)$ or simply $f\ 2$
- The latter notation has an advantage:
  - Anonymous functions: $\backslash x \rightarrow x * x$ – Haskell syntax

More about writing function application

- Classical: $f(2)$
- Available: $(f\ 2)$ or simply $f\ 2$
- The latter notation has an advantage:
  - Anonymous functions: $\backslash x \rightarrow x * x$ – Haskell syntax
  - Application: $(\backslash x \rightarrow x * x)\ 2$

Computation as reduction

- *Reduction*: the *evaluation* (i.e. transform complex expression into its value) is done via *rewriting*

Computation as reduction

- *Reduction*: the *evaluation* (i.e. transform complex expression into its value) is done via *rewriting*
- Haskell: `sum n = if n <= 0 then 0 else n + sum (n - 1)`

$$
\begin{aligned}
\texttt{sum 2} &\to \texttt{(if n <= 0 then 0 else n + sum (n - 1)) 2} \\
&\to \texttt{if 2 <= 0 then 0 else 2 + sum (2 - 1)} \\
&\to \texttt{2 + sum (2 - 1)} \\
&\to \texttt{2 + sum 1} \\
&\to \texttt{2 + (if n <= 0 then 0 else n + sum (n - 1)) 1} \\
&\to \texttt{2 + (if 1 <= 0 then 0 else 1 + sum (1 - 1))} \\
&\to \texttt{2 + (1 + sum (1 - 1))} \\
&\to \texttt{2 + (1 + sum 0)} \\
&\to \texttt{2 + (1 + (if n <= 0 then 0 else n + sum (n - 1) 0))} \\
&\to \texttt{2 + (1 + (if 0 <= 0 then 0 else 0 + sum (0 - 1)))} \\
&\to \texttt{2 + (1 + 0)} \\
&\to \texttt{2 + 1} \\
&\to \texttt{3}
\end{aligned}
$$

Fundamental ingredients

Syntax:

- no commands, only expressions

Fundamental ingredients

Syntax:

- no commands, only expressions
- primitive (builtin) values and operators, the conditional expression

Fundamental ingredients

Syntax:

- no commands, only expressions
- primitive (builtin) values and operators, the conditional expression
- *Abstraction*: given expression *exp* and identifier x we can construct an expression $(\backslash x \rightarrow exp)$

Fundamental ingredients

Syntax:

- no commands, only expressions
- primitive (builtin) values and operators, the conditional expression
- *Abstraction*: given expression *exp* and identifier x we can construct an expression $(\backslash x \rightarrow exp)$
  - *exp* is “abstracted” from the specific value bound to x

Fundamental ingredients

Syntax:

- no commands, only expressions
- primitive (builtin) values and operators, the conditional expression
- *Abstraction*: given expression *exp* and identifier x we can construct an expression $(\backslash x \rightarrow exp)$
  - *exp* is “abstracted” from the specific value bound to x
- *Application*: $f\ a$

Fundamental ingredients

Syntax:

- no commands, only expressions
- primitive (builtin) values and operators, the conditional expression
- *Abstraction*: given expression *exp* and identifier x we can construct an expression $(\backslash x \rightarrow exp)$
  - *exp* is “abstracted” from the specific value bound to x
- *Application*: $f\ a$
- No constraints on the possibility of passing functions as arguments or returning them!

Fundamental ingredients

Syntax:

- no commands, only expressions
- primitive (builtin) values and operators, the conditional expression
- *Abstraction*: given expression *exp* and identifier x we can construct an expression $(\backslash x \rightarrow exp)$
  - *exp* is “abstracted” from the specific value bound to x
- *Application*: $f\ a$
- No constraints on the possibility of passing functions as arguments or returning them!
  DEMO

Fundamental ingredients

Semantics:

- *Evaluation*: *reduction*

Fundamental ingredients

Semantics:

- *Evaluation*: *reduction*
- Two operations

Fundamental ingredients

Semantics:

- *Evaluation*: *reduction*
- Two operations:
  - simple search: when an id is bound in the environment, replace it by its definition

Fundamental ingredients

Semantics:

- *Evaluation*: *reduction*
- Two operations:
  - simple search: when an id is bound in the environment, replace it by its definition
  - β-rule: functional expression applied to an argument

Fundamental ingredients

Semantics:

- *Evaluation*: *reduction*
- Two operations:
  - simple search: when an id is bound in the environment, replace it by its definition
  - β-rule: functional expression applied to an argument
- *Redex*: is an application of the form $(\backslash\ x \rightarrow x + 1)\ 3$

Fundamental ingredients

Semantics:

- *Evaluation*: *reduction*
- Two operations:
  - simple search: when an id is bound in the environment, replace it by its definition
  - β-rule: functional expression applied to an argument
- *Redex*: is an application of the form $(\backslash\ x \rightarrow x + 1)\ 3$
- *Reductum*: the reductum of a redex is the expression obtained by replacing in body each free occurrence of the parameter

Fundamental ingredients

Semantics:

- *Evaluation*: *reduction*
- Two operations:
  - simple search: when an id is bound in the environment, replace it by its definition
  - β-rule: functional expression applied to an argument
- *Redex*: is an application of the form $(\backslash\ x \rightarrow x + 1)\ 3$
- *Reductum*: the reductum of a redex is the expression obtained by replacing in body each free occurrence of the parameter

Examples:

Fundamental ingredients

Semantics:

- *Evaluation*: *reduction*
- Two operations:
  - simple search: when an id is bound in the environment, replace it by its definition
  - β-rule: functional expression applied to an argument
- *Redex*: is an application of the form $(\backslash\ x \rightarrow x + 1)\ 3$
- *Reductum*: the reductum of a redex is the expression obtained by replacing in body each free occurrence of the parameter

Examples: 3 + 1

Fundamental ingredients

Semantics:

- *Evaluation*: *reduction*
- Two operations:
  - simple search: when an id is bound in the environment, replace it by its definition
  - β-rule: functional expression applied to an argument
- *Redex*: is an application of the form $(\backslash\ x \rightarrow x + 1)\ 3$
- *Reductum*: the reductum of a redex is the expression obtained by replacing in body each free occurrence of the parameter

Examples: 3 + 1

```
sum 2 →   (if n <= 0 then 0 else n + sum (n − 1)) 2
      →        if 2 <= 0 then 0 else 2 + sum (2 − 1)
  ...
```

Evaluation

Haskell example:

```
doubleMe x = 2 * x
tripleMe y = 3 * y
compose f g = f . g
```

Evaluation

Haskell example:

```
doubleMe x = 2 * x
tripleMe y = 3 * y
compose f g = f . g
```

How to evaluate:

`((compose doubleMe) tripleMe) (doubleMe 4)` ?

Evaluation

Haskell example:

```
doubleMe x = 2 * x
tripleMe y = 3 * y
compose f g = f . g
```

How to evaluate:
`((compose doubleMe) tripleMe) (doubleMe 4)` ?

Strategies:

- Evaluation by value
- Evaluation by name
- Lazy evaluation

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:
   `(compose doubleMe tripleMe) (doubleMe 4)`

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:
   `(compose doubleMe tripleMe) (doubleMe 4)`
2. Evaluate `(compose doubleMe tripleMe)`:
   `(\x → 2 * ((\y → 3 * y ) x))`

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:
   `(compose doubleMe tripleMe) (doubleMe 4)`
2. Evaluate `(compose doubleMe tripleMe)`:
   `(\x → 2 * ((\y → 3 * y ) x))`

   Note: we have obtained a **value** of *functional type*

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:
   `(compose doubleMe tripleMe) (doubleMe 4)`
2. Evaluate `(compose doubleMe tripleMe)`:
   `(\x → 2 * ((\y → 3 * y ) x))`

   Note: we have obtained a **value** of *functional type*
3. Evaluate `(doubleMe 4)` : `8`

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:
   `(compose doubleMe tripleMe) (doubleMe 4)`
2. Evaluate `(compose doubleMe tripleMe)`:
   `(\x → 2 * ((\y → 3 * y ) x))`

   Note: we have obtained a **value** of *functional type*
3. Evaluate `(doubleMe 4)` : `8`

   Note: we have obtained a **value** of *primitive type*

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:
   `(compose doubleMe tripleMe) (doubleMe 4)`
2. Evaluate `(compose doubleMe tripleMe)`:
   `(\x → 2 * ((\y → 3 * y ) x))`

   Note: we have obtained a **value** of *functional type*
3. Evaluate `(doubleMe 4)` : `8`

   Note: we have obtained a **value** of *primitive type*
4. Reduce the redex:
   `(\x → 2 * ((\y → 3 * y ) x)) 8`

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:
   `(compose doubleMe tripleMe) (doubleMe 4)`
2. Evaluate `(compose doubleMe tripleMe)`:
   `(\x → 2 * ((\y → 3 * y ) x))`

   Note: we have obtained a **value** of *functional type*
3. Evaluate `(doubleMe 4)` : `8`

   Note: we have obtained a **value** of *primitive type*
4. Reduce the redex:
   `(\x → 2 * ((\y → 3 * y ) x)) 8`
5. Go to step 1.

Complete reduction

- Consider the redex:

```
(\x → 2 * ((\y → 3 * y ) x)) 8
```

Complete reduction

- Consider the redex:

```
(\x -> 2 * ((\y -> 3 * y ) x)) 8
```

- To reduce this, we apply the β-rule twice:

```
2 * ((\y -> 3 * y ) 8)
```

Complete reduction

- Consider the redex:

```
(\x → 2 * ((\y → 3 * y ) x)) 8
```

- To reduce this, we apply the β-rule twice:

```
2 * ((\y → 3 * y ) 8)
```

```
2 * (3 * 8)
```

Complete reduction

- Consider the redex:

```
(\x -> 2 * ((\y -> 3 * y ) x)) 8
```

- To reduce this, we apply the β-rule twice:

```
2 * ((\y -> 3 * y ) 8)

2 * (3 * 8)

48
```

Evaluation by name

Steps:

1. Scan expression from left to right and find the first application, say:
   `(compose doubleMe tripleMe) (doubleMe 4)`

Evaluation by name

Steps:

1. Scan expression from left to right and find the first application, say:
   `(compose doubleMe tripleMe) (doubleMe 4)`
2. Evaluate `(compose doubleMe tripleMe)`:
   `(\x → 2 * ((\y → 3 * y ) x))`

Evaluation by name

Steps:

1. Scan expression from left to right and find the first application, say:
   `(compose doubleMe tripleMe) (doubleMe 4)`
2. Evaluate `(compose doubleMe tripleMe)`:
   `(\x → 2 * ((\y → 3 * y ) x))`

   Note: we have obtained a **value** of *functional type*

Evaluation by name

Steps:

1. Scan expression from left to right and find the first application, say:
   `(compose doubleMe tripleMe) (doubleMe 4)`
2. Evaluate `(compose doubleMe tripleMe)`:
   `(\x → 2 * ((\y → 3 * y ) x))`

   Note: we have obtained a **value** of *functional type*
3. Reduce the redex using β-reduction:
   `(\x→ 2 * ((\y → 3 * y ) x)) (doubleMe 4)`
4. Go to step 1

Lazy evaluation

- The call by name's problem: duplicate expressions introduce when performing β-reduction in evaluation by name (step 3).

Lazy evaluation

- The call by name's problem: duplicate expressions introduce when performing β-reduction in evaluation by name (step 3).
- Imagine that `(doubleMe 4)` is used at least twice

Lazy evaluation

- The call by name's problem: duplicate expressions introduce when performing β-reduction in evaluation by name (step 3).
- Imagine that `(doubleMe 4)` is used at least twice$\rightarrow$ inefficient evaluations introduced

Lazy evaluation

- The call by name's problem: duplicate expressions introduce when performing β-reduction in evaluation by name (step 3).
- Imagine that `(doubleMe 4)` is used at least twice$\rightarrow$ inefficient evaluations introduced
- Lazy evaluation: when `(doubleMe 4)` is evaluated the first time it keeps a copy and uses it later on

Lazy evaluation

- The call by name's problem: duplicate expressions introduce when performing β-reduction in evaluation by name (step 3).
- Imagine that `(doubleMe 4)` is used at least twice$\rightarrow$ inefficient evaluations introduced
- Lazy evaluation: when `(doubleMe 4)` is evaluated the first time it keeps a copy and uses it later on
- Evaluation by name + lazy evaluation = *call by need* strategies

Resources

Please refer to the book:

- Chapter 11: `http://websrv.dthu.edu.vn/attachments/newsevents/content2415/Programming_Languages_-_Principles_and_Paradigms_thereds1106.pdf`

in order to grasp other related items:

1. λ- calculus
2. local environment
3. types
4. pattern matching
5. infinite objects